

Application Security: OWASP Top 10 & API Risks

Session 6 - The Application Layer





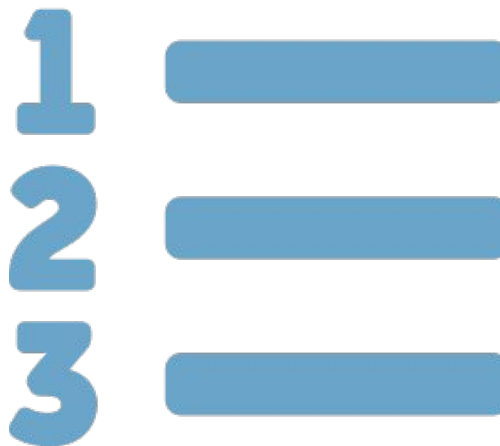
S6 Application Security: OWASP Top 10 & API Risks



Session Overview

The vocabulary for evaluating what you already understand

- **LO2 - Evaluate different approaches to assessing a system's security, and use these to evaluate the security of a system**
- **LO4 - Develop and evaluate software that addresses the most common and most severe security concerns**



You Already Met This Breach



- **Session 1: Equifax - 147 million records exposed, mapped across all five properties**
- **The technical cause was one flaw in Apache Struts2, CVE-2017-5638 - an unpatched web framework component**
- **By the end of today you can name that flaw precisely: OWASP A03 - Injection**
- **This is the shift Session 6 makes - not new knowledge, a name for what you can already reason about**

OWASP Is a Vocabulary, Not Ten New Things

- 
- You can already reason about broken access control (S5), weak crypto (S3), and weak authentication (S4)
 - OWASP Top 10 gives these failures precise, industry-standard names
 - Naming precisely means you can evaluate a real system and cite a category, not write a vague vulnerability
 - This precision is exactly what Part B (B4) rewards - specific citation over general description

Analogy

You already recognise the symptoms of a fever. Today you learn to write 'pyrexia' on the chart, so any other evaluator instantly knows what you mean.

Depth Is Not Equal Across the Ten



Four Deep - Extends What You Know

- A01 Broken Access Control - extends Session 5
- A02 Cryptographic Failures - extends Session 3
- A03 Injection - the one genuinely new mechanism
- A07 Identification & Authentication Failures - extends Session 4

Two Brief - Forward Pointers

- A05 Security Misconfiguration - points forward to S7 and S10
- A09 Logging & Monitoring Failures - points forward to S11
- OWASP API Top 5 is covered separately as a brief comparison, not a deep teach

A01 - Broken Access Control

- 
- **Extends Session 5 - authorisation, least privilege, and the IDOR lab you already ran**
 - **Root cause: the system checks who you are, but not what you are allowed to touch**
 - **Includes IDOR, privilege escalation, and reaching another user's resource by changing a URL or ID**
 - **STRIDE link: this is Elevation of Privilege - violates Authenticity + Accountability**

Analogy

Like a hotel keycard that opens every room on the floor, not just yours - the door checked that you had A key, not that you had YOUR key.

A01 - When It Fails



- **Failure: change a URL parameter (e.g. an account ID) and view another user's private records**
- **Property violated: Accountability (no one authorised this access) + Authenticity (system cannot confirm this is a legitimate action for this user)**
- **Layer: Application**
- **SRS hook: APP-01 (input validation and authorisation checks) - feeds Part B4**

A02 - Cryptographic Failures



- **Extends Session 3 - the crypto properties you already mapped to use cases**
- **Includes weak or missing encryption, weak password hashing (MD5/SHA-1 instead of bcrypt/Argon2), hardcoded secrets, poor key management**
- **Root cause: the right property (Confidentiality or Authenticity) was skipped, or implemented with a broken mechanism**

Analogy

Like locking your door with a padlock a bolt-cutter opens in seconds - a lock exists, but it does not do the job of locking.

A02 - When It Fails



- **Failure: a password database is compromised, but hashed with unsalted MD5 - cracked in hours on commodity hardware**
- **Property violated: Confidentiality + Authenticity**
- **Layer: Data (storage), Application (where hashing happens), Network (if TLS is weak or absent)**
- **SRS hook: APP-01, DATA-01/DATA-02 - feeds Part B4 and B5**

A03 - Injection: The One Genuinely New Idea

- 
- **SQL, command, and LDAP injection - different languages, the same underlying flaw**
 - **Root cause: untrusted input is allowed to change the STRUCTURE of a query or command, not just its data**
 - **Violates Integrity, Confidentiality, AND Availability - one of the oldest vulnerability classes, still consistently in the top three**

Analogy

Like a mail-merge letter where the [NAME] field lets someone write their own instruction into the letter - because the system trusted that field to only ever contain a name.

Same Root Cause, Three Languages

SQL Injection	Command Injection	LDAP Injection
SQL	CMD	LDAP
Untrusted input alters a database query - e.g. bypasses a login check or extracts an entire table.	Untrusted input is passed to an OS shell - e.g. a filename field ends up executing a system command.	Untrusted input alters a directory-service query - e.g. bypasses authentication against a corporate directory.

Same root cause every time: input becomes structure, not just data

KEY CONCEPT

It Is Not 'Bad Input.' It Is Input That Becomes Structure.

A query or command has two parts: the fixed structure the developer wrote, and the data the user supplies. Injection happens when user-supplied data is allowed to change the structure itself - a name field becomes part of the SQL statement, a filename becomes part of the shell command. The fix is never just 'validate harder' - it is keeping data and structure permanently separate: parameterised queries, escaping, allow-lists.

A03 - When It Fails



- **Real case: CVE-2017-5638 (Apache Struts2, CVSS 10.0) - an HTTP header treated as executable code, exploited in the Equifax breach**
- **Layer: Application (root cause); can cascade into Data (Confidentiality), Integrity, and Availability**
- **SRS hook: APP-01 (input validation, parameterised queries) - feeds Part B4**

Try it now: Query Structure Sandbox demo - type into the search box and watch the query break.

A07 - Identification & Authentication Failures



- **Extends Session 4 - password storage, session cookies, and JWTs**
- **Includes weak passwords permitted, no MFA offered, insecure session management, credential exposure**
- **Property violated: Authenticity - the system can no longer trust that the entity is who it claims to be**

Analogy

Like a bouncer who checks that SOMEONE showed ID, but never actually reads the name or photo on it.

A07 - When It Fails



- **Failure: no MFA on an admin account - a leaked password alone is enough to log in**
- **Property violated: Authenticity**
- **Layer: IAM (the authentication mechanism itself), Application (session handling)**
- **SRS hook: IAM-01 (MFA), APP-01 - feeds Part B2 and B4**

Six Categories, One Evaluation Lens

4

Deep categories

extend S3-S5

2

Brief categories

forward pointers

3

API risks

comparison only

Category	Property Violated	Layer	SRS Class
A01 Broken Access Control	Accountability + Authenticity	Application	APP-01
A02 Cryptographic Failures	Confidentiality + Authenticity	Data / Application	APP-01, DATA-01/02
A03 Injection	Integrity + Confidentiality + Availability	Application	APP-01
A07 Authentication Failures	Authenticity	IAM / Application	IAM-01, APP-01
A05 Misconfiguration (brief)	varies by control	System / Network	SYS-01, NET-0x (S7, S10)
A09 Logging Failures (brief)	Accountability	All (observability)	(designed in S11)

Two Categories You Will See Again



A05 - Security Misconfiguration (Brief)

- Default credentials left active, unnecessary features enabled, missing security headers, overpermissioned cloud storage
- Forward pointer: hardening mechanics are Session 7 (System layer); configuration-review tooling is Session 10

A09 - Logging & Monitoring Failures (Brief)

- Absence of logging, insufficient alerting, no audit trail - directly violates Accountability
- Forward pointer: designing observability as a property, not an afterthought, is Session 11
- Reinforces Session 1: Accountability was named as a property from day one, not bolted on later

OWASP API Top 5 - A Brief Comparison

Object-Level Authorisation API-01	Excessive Data Exposure API-03	Lack of Rate Limiting API-04
Broken Object Level Authorisation (BOLA) - same root cause as IDOR (A01), applied to an API endpoint instead of a web page.	The API returns more fields than the client needs - the extra data is a Confidentiality risk even if the UI hides it.	No limit on request volume - maps directly to NET-02 in the SRS (an Availability control).

Comparison only - full API security treatment is outside Session 6's scope

Activity - CVE Mapping



1. Classify each CVE under the correct OWASP Top 10 category
2. Identify the root cause
3. Name the security property violated
4. State which SRS requirement class would have been relevant (APP-01, IAM-01, etc.)
5. Discuss: which of these could appear in your assigned coursework domain?

- CVE-2019-11510 - Pulse Secure VPN
- CVE-2014-0160 - Heartbleed (OpenSSL)
- CVE-2017-5638 - Apache Struts2
- CVE-2020-1472 - Zerologon

Small Lab - WebGoat / DVWA

1. One guided SQL injection exercise and one access-control bypass
2. Observe the vulnerability - do not write the exploit from scratch
3. Identify what the developer should have done differently
4. Draft a finding: title, severity, evidence, OWASP category, SRS mapping



Goal: recognition and structured documentation - not attack craft.

The Payoff: Before and After Session 6



Before S6

- 'The login seems insecure.'
- 'There might be a database problem.'
- Vague, hard to grade, hard to defend in a viva

After S6

- 'A07 - no MFA on the admin login; violates Authenticity; IAM-01 unmet.'
- 'A03 Injection - untrusted input reaches the query structure; APP-01 unmet.'
- Specific, citable, defensible - this is what Part B4 is graded on

Questions & Answers

*Session 6 - Application Security: OWASP Top 10 & API
Risks*



Disclaimer & Copyrights



The information presented in this presentation is intended for general informational purposes only. It is not meant to be comprehensive or to constitute professional advice. Any reliance you place on such information is strictly at your own risk. While I strive to keep the information accurate and up-to-date, I do make no representations or warranties of any kind, express or implied, about the completeness, accuracy, reliability, suitability, or availability concerning the content within this presentation.

This presentation may include references or links to third-party websites or content. I do not endorse the views expressed or the information provided on such external sites. I am not responsible for the content of any linked site or any link contained in a linked site.

Any action you take upon the information presented in this presentation is strictly at your own discretion. I will not be liable for any losses or damages in connection with the use of this information.

This disclaimer is subject to change without notice.

@ Images and artworks used in this presentation are used for general informational purposes only. It may contain copyrighted material, the use of which may not have been specifically authorised by the copyright owner. This material is available in an effort to explain issues relevant to the topic. This should constitute a 'fair use' of any such copyrighted material.

Last modified: 3 Jul 2026